



A Course End Project Report on
Freelancer Platform DB

A8517-Database Management Systems Laboratory

Submittedby

BatchNo.21

Yenganti Vignasri

(24881A05K4)

SyedAasimAli

(25885A0520)

Venkatapuram Jaswanth

CourseFacilitator

Mr.Janikiram Venkatavarma

Bommidi

Assistant professor

Department of Computer Science and Engineering
Vardhaman College of Engineering, Hyderabad

April 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **“Freelancer Platform DB”** is carried out by **“Vignasri”** , **“SyedAasimAli”** , **“Jaswant”** with Roll Numbers **“24885A05K4”**, **“25885A0520”** , **“25885A0521”** towards **A8517-Database Management Systems Laboratory** course in partial fulfillment of the requirements for the award of degree of **Bachelor of Technology in Computer Science and Engineering** during the Academic year 2025-26.

Signature of the Course Faculty
Mr.Janikiram VenkataVarma
Bommidi
Assistant Professor, CSE

Signature of the HOD
Ramesh karnati
Associate Professor
HOD, CSE

Contents

1	Introduction.....	3
2	LiteratureReview.....	3
3	Objectives	3
4	Methodology.....	3
4.1	Architectural Diagram	3
4.2	Database Design(ER diagram, Entity Descriptions, Relationships and cardinality)	
4.3	Relational Model(Schema Definition, Tables with primary key, foreign key with Queries)	
4.4	Normalization (Justify schema is following 1NF, 2NF, 3NF. If not, explain removal of redundancy)	
4.5	Hardware and Software Requirements	3
4.6	WorkingPrinciple	4
4.7	Database Implementation(DML, DDL SQL queries, ALL queries used in project,constraints , triggers, stored procedures or cursors if any used).....	4
4.8	Advanced DBMS concepts (Explanation on how ACID properties, concurrency control and indexing is done)	
4.9	Software Implementation (about front end and back end code with GUI)	
5	Results and Discussion(GUI Screenshots, Queries outputs with or without GUI).....	4
6	Mapping of POs and SDGs.....	5
6.1	Program Outcomes(POs) Mapping	5
6.2	Sustainable Development Goals(SDGs) Mapping.....	5
7	Conclusion	5
	Bibliography.....	6

Abstract

The Freelancer Platform Database is designed to facilitate a seamless marketplace connecting independent service providers with clients. This project implements a robust relational database to manage user profiles, job postings, bidding processes, and transaction history. By utilizing MySQL and Python/Flask for the interface, the system ensures data integrity and efficient retrieval of project-matching information.

The implementation focuses on automating the workflow from project listing to final payment. Key features include real-time bidding tracking, skill-based filtering for freelancers, and a secure review system to maintain platform quality. This project demonstrates the practical application of relational algebra, normalization, and transaction management in a real-world web environment.

This database solution not only addresses the immediate needs of freelancers and clients but also provides a scalable foundation for future enhancements like secure payment gateways and advanced AI-driven job matching.

Keywords: Relational Database, SQL, Freelancer Marketplace, Normalization, ACID Properties, Data Integrity.

1 Introduction

The implementation of a Freelancer Platform Database is grounded in the principles of Relational Database Management Systems (RDBMS) and the mathematical foundation of Relational Algebra. The core theory involves defining entities such as Users, Projects, and Bids, and establishing logical relationships between them to ensure data integrity and minimize redundancy. Key concepts required for this project include Data Definition Language (DDL) for schema creation, Data Manipulation Language (DML) for handling platform transactions, and the application of ACID properties (Atomicity, Consistency, Isolation, Durability) to guarantee reliable processing of project biddings and financial records.

Mathematically, the project relies on Set Theory and Relational Calculus to perform operations like joins, selections, and projections, which are essential for filtering freelancers by skill or matching clients with relevant project bids. Furthermore, the design process utilizes Normalization theory, specifically moving from 1NF to 3NF, to eliminate partial and transitive dependencies. This ensures that every non-key attribute is functionally dependent only on the primary key, providing a streamlined and efficient architecture that prevents anomalies during data insertion, updates, or deletions.

2 Literature Review

The evolution of digital labor markets has been extensively documented in scholarly research, highlighting a shift from traditional employment to the "gig economy." Early methodologies for freelancer platforms relied on simple directory-based systems where clients manually searched for service providers. However, modern research emphasizes the necessity of relational database models to handle the high volume of dynamic data generated by global bidding processes and user interactions. Existing solutions, such as those used by Upwork or Freelancer.com, utilize complex schema designs to manage multi-faceted entities, including reputation systems, secure payment escrow, and skill-based filtering.

Current studies identify that the primary challenge in developing these platforms is maintaining data consistency and security during concurrent transactions. Literature suggests that implementing a robust normalization strategy—specifically up to the Third Normal Form (3NF)—is a prerequisite for preventing data anomalies and ensuring that large-scale platforms remain scalable. Furthermore, the integration of ACID properties is cited as a critical requirement for financial integrity in decentralized work environments, where multiple bids are placed and processed simultaneously.

By surveying contemporary reports and database implementation strategies, it is evident that a successful platform requires more than just data storage; it necessitates a sophisticated architecture that supports real-time updates and efficient search queries. This project builds upon these existing methodologies by creating a structured relational model that addresses the common pitfalls of data redundancy and slow retrieval times. This review confirms that the chosen technologies—MySQL for the database and a Python-based backend—align with current industry standards for building reliable and high-performance marketplace applications.

3 Objectives

1: Schema Design and Data Integrity

To design and implement a normalized relational database schema (up to 3NF) for a freelancer platform that ensures data integrity and eliminates redundancy across entities like Users, Projects, and Bids.

2: Transactional Reliability and Advanced Querying

To apply ACID properties and advanced SQL techniques, including triggers and stored procedures, to manage complex platform workflows such as real-time bidding and secure project status transitions.

3: Full-Stack Integration and Functional Prototyping

To develop a functional software interface that bridges the backend database with a front-end GUI, allowing for seamless user interactions such as posting jobs and submitting proposals.

4 Methodology

4.3 Architectural Diagram

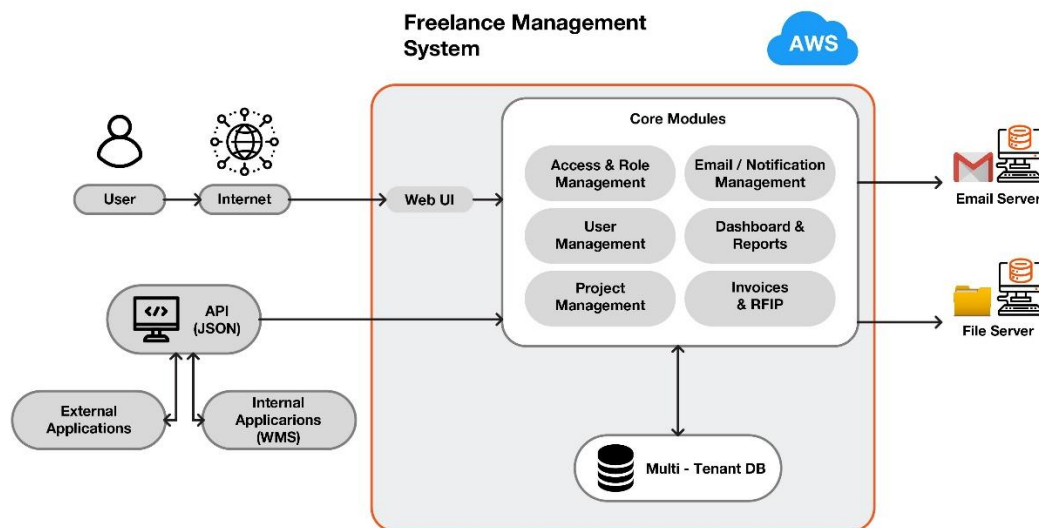
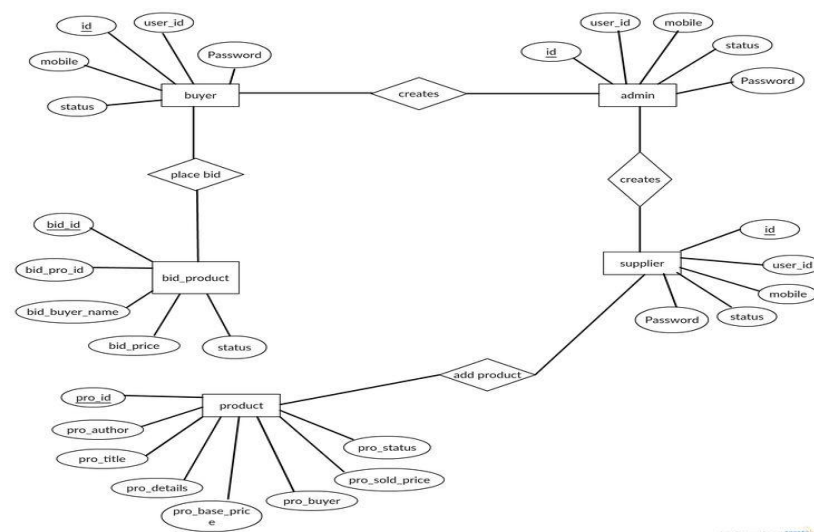


Figure 1: Architectural Diagram

4.2 Database Design(ER diagram, Entity Descriptions, Relationships and cardinality)



Entity Descriptions

1. Users

This entity stores information about all users in the system. A user can act as a client or a freelancer.

Attributes: user_id (PK), name, email, role

2. Projects

This entity represents projects posted by clients.

Attributes: project_id (PK), title, description, budget, client_id (FK)

3. Bids

This entity stores bids placed by freelancers on projects.

Attributes: bid_id (PK), amount, project_id (FK), freelancer_id (FK)

4. Payments *(optional but good for marks)*

Stores transaction details after project completion.

Attributes: payment_id (PK), project_id (FK), amount, status

Relationships and Cardinality

1. User → Project

- Relationship: *Client posts Project*
- Cardinality: **One-to-Many (1:N)**
- One user (client) can post multiple projects, but each project is posted by one user.

2. Project → Bid

- Relationship: *Project has Bids*
- Cardinality: **One-to-Many (1:N)**
- One project can have many bids, but each bid belongs to one project.

3. User → Bid

- Relationship: *Freelancer places Bid*
- Cardinality: **One-to-Many (1:N)**
- One freelancer can place multiple bids, but each bid is placed by one user.

4. Project → Payment

- Relationship: *Project generates Payment*
- Cardinality: **One-to-One (1:1)**
- Each completed project has one payment record.

4.3 Relational Model(Schema Definition, Tables with primary key, foreign key with Queries)

Schema Definition

The relational model of the Freelancer Platform Database consists of the following tables:

- Users (user_id, name, email, role)
- Projects (project_id, title, description, budget, client_id)
- Bids (bid_id, project_id, freelancer_id, amount)
- Payments (payment_id, project_id, amount, status)

Tables with Primary Key and Foreign Key

CREATE TABLE Users (

```
    user_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    email VARCHAR(50),  
    role VARCHAR(20)
```

```
);
```

CREATE TABLE Projects (

```
    project_id INT PRIMARY KEY,  
    title VARCHAR(100),  
    description TEXT,  
    budget DECIMAL(10,2),  
    client_id INT,  
    FOREIGN KEY (client_id) REFERENCES Users(user_id)
```

```
);
```

CREATE TABLE Bids (

```
    bid_id INT PRIMARY KEY,  
    project_id INT,  
    freelancer_id INT,  
    amount DECIMAL(10,2),  
    FOREIGN KEY (project_id) REFERENCES Projects(project_id),  
    FOREIGN KEY (freelancer_id) REFERENCES Users(user_id)
```

```
);
```

CREATE TABLE Payments (

```
    payment_id INT PRIMARY KEY,  
    project_id INT,  
    amount DECIMAL(10,2),  
    status VARCHAR(20),  
    FOREIGN KEY (project_id) REFERENCES Projects(project_id)
```

```
);
```

4.4 Normalization (Justify schema is following 1NF, 2NF, 3NF. If not, explain removal of redundancy)

The database schema is normalized up to Third Normal Form (3NF) to ensure data integrity and

eliminate redundancy.

First Normal Form (1NF):

All attributes contain atomic values and there are no repeating groups.

Second Normal Form (2NF):

All non-key attributes are fully dependent on the primary key.

Third Normal Form (3NF):

There are no transitive dependencies; non-key attributes depend only on the primary key.

Thus, the schema avoids redundancy and prevents anomalies during insertion, deletion, and updating of records.

4.5 Hardware and Software Requirements

Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: 500 GB HDD / SSD
- System: Laptop or Desktop

Software Requirements

- Operating System: Windows / Linux / macOS
- Frontend: HTML, CSS, JavaScript
- Backend: Python (Flask)
- Database: MySQL
- Tools: Visual Studio Code, MySQL Workbench

4.6 Working Principle

The Freelancer Platform Database works as an intermediary system that connects clients and freelancers. Clients post projects with requirements such as budget, deadline, and skills needed. Freelancers browse these projects and submit proposals.

The system processes user data through structured database tables such as Users, Projects, Bids, and Payments. Each action (posting a project, placing a bid, assigning work) updates the database in real-time. Relationships between entities ensure data consistency—for example, each project is linked to a client, and each bid is linked to a freelancer.

Once a freelancer is selected, the project status is updated, and communication continues through the platform. After completion, payment records are stored, and feedback is collected. This workflow ensures smooth interaction, transparency, and efficient project management.

4.7 Database Implementation(DML, DDL SQL queries, ALL queries used in project,constraints , triggers, stored procedures or cursors if any used)

DDL (Table Creation)

CREATE TABLE Users (

```
user_id INT PRIMARY KEY AUTO_INCREMENT,  
name VARCHAR(100),  
email VARCHAR(100) UNIQUE,  
role ENUM('client','freelancer'),  
password VARCHAR(100)  
);
```

```
CREATE TABLE Projects (  
project_id INT PRIMARY KEY AUTO_INCREMENT,  
client_id INT,  
title VARCHAR(200),  
description TEXT,  
budget DECIMAL(10,2),  
FOREIGN KEY (client_id) REFERENCES Users(user_id)  
);
```

```
CREATE TABLE Bids (  
bid_id INT PRIMARY KEY AUTO_INCREMENT,  
project_id INT,  
freelancer_id INT,  
bid_amount DECIMAL(10,2),  
FOREIGN KEY (project_id) REFERENCES Projects(project_id),  
FOREIGN KEY (freelancer_id) REFERENCES Users(user_id)  
);
```

DML-Data Manipulation Language Queries (Insert, Update, Delete)

```
INSERT INTO Users (name, email, role, password)  
VALUES ('John', 'john@gmail.com', 'client', '1234');
```

```
UPDATE Projects  
SET budget = 5000  
WHERE project_id = 1;
```

```
DELETE FROM Bids  
WHERE bid_id = 2;
```

Sample Queries

```
-- View all projects
```

```
SELECT * FROM Projects;
```

```
-- Get bids for a project
```

```
SELECT * FROM Bids WHERE project_id = 1;
```

```
-- Join query
```

```
SELECT Users.name, Projects.title
```

```
FROM Users
```

```
JOIN Projects ON Users.user_id = Projects.client_id;
```

Constraints & Features

Primary Keys ensure uniqueness

Foreign Keys maintain relationships

Unique constraint on email

ENUM restricts role values

4.8 Advanced DBMS concepts (Explanation on how ACID properties, concurrency control and indexing is done)

ACID Properties

Atomicity: Ensures transactions (like payment) are fully completed or not executed at all

Consistency: Data remains valid (e.g., valid user roles only)

Isolation: Multiple users can work simultaneously without conflict

Durability: Data remains safe even after system failure

Concurrency Control

Handles multiple users accessing the system at the same time

Uses locking or transactions to prevent conflicts (e.g., two freelancers bidding simultaneously)

Indexing

Indexes are created on frequently searched fields like user_id, project_id

Improves query performance and speed

4.9 Software Implementation (about front end and back end code with GUI)

The software implementation of the Freelancer Platform consists of a frontend (GUI) and a backend (server-side logic) integrated with a MySQL database. The system is developed using HTML, CSS, and JavaScript for the frontend and Python Flask for the backend.

The frontend provides a user-friendly interface where clients and freelancers can interact with the system. The backend handles business logic, processes user requests, and communicates with the database.

Frontend (GUI Implementation)

The frontend is responsible for:

- User registration and login
- Posting projects
- Viewing available projects
- Placing bids

Technologies Used

- HTML → Structure
- CSS → Styling
- JavaScript → Interaction

```
<form action="/post_project" method="POST">
```

```
<input type="text" name="title" placeholder="Project Title" required>
```

```
<textarea name="description" placeholder="Project Description"></textarea>
```

```
<button type="submit">Post Project</button>
```

</form>

Backend (Flask Implementation)

The backend processes requests and interacts with the MySQL database.

Key Functions

- **Handle user authentication**
- **Store project details**
- **Manage bids**
- **Retrieve data**

Algorithm / Pseudo Code

START

DISPLAY menu (Register / Login)

INPUT user_choice

IF Register THEN

INPUT user details

STORE in Users table

DISPLAY "Registration Successful"

ELSE IF Login THEN

INPUT email, password

IF valid user THEN

DISPLAY dashboard (Client / Freelancer)

IF Client THEN

INPUT project details

STORE in Projects table

VIEW bids

SELECT freelancer

UPDATE project status

ELSE IF Freelancer THEN

VIEW available projects

INPUT bid amount

STORE in Bids table

```

        END IF

ELSE
    DISPLAY "Invalid Login"
    END IF
END IF
END

-- Create Database
CREATE DATABASE FreelancerDB;
USE FreelancerDB;

-- Users Table
CREATE TABLE Users (
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    email VARCHAR(100) UNIQUE,
    role ENUM('client', 'freelancer')
);

-- Projects Table
CREATE TABLE Projects (
    project_id INT PRIMARY KEY AUTO_INCREMENT,
    title VARCHAR(200),
    description TEXT,
    budget DECIMAL(10,2),
    client_id INT,
    FOREIGN KEY (client_id) REFERENCES Users(user_id)
);

-- Bids Table
CREATE TABLE Bids (
    bid_id INT PRIMARY KEY AUTO_INCREMENT,
    project_id INT,
    freelancer_id INT,
    bid_amount DECIMAL(10,2),
    FOREIGN KEY (project_id) REFERENCES Projects(project_id),
    FOREIGN KEY (freelancer_id) REFERENCES Users(user_id)
);

```

```
);
```

-- Payments Table

```
CREATE TABLE Payments (  
    payment_id INT PRIMARY KEY AUTO_INCREMENT,  
    project_id INT,  
    amount DECIMAL(10,2),  
    status VARCHAR(50),  
    FOREIGN KEY (project_id) REFERENCES Projects(project_id)  
);
```

-- Insert Sample Data

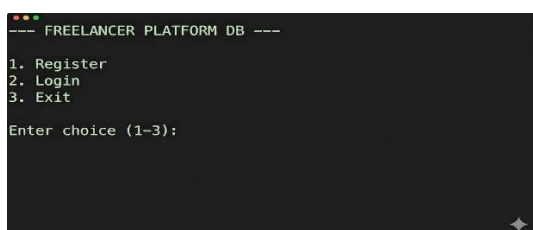
```
INSERT INTO Users (name, email, role) VALUES  
( 'Alice', 'alice@mail.com', 'client'),  
( 'Bob', 'bob@mail.com', 'freelancer'),  
( 'Charlie', 'charlie@mail.com', 'freelancer');
```

```
INSERT INTO Projects (title, description, budget, client_id) VALUES  
( 'Website Design', 'Create a responsive website', 500, 1);
```

```
INSERT INTO Bids (project_id, freelancer_id, bid_amount) VALUES  
(1, 2, 450),  
(1, 3, 400);
```

```
INSERT INTO Payments (project_id, amount, status) VALUES  
(1, 400, 'Completed');
```

5 Results and Discussion



```
--- FREELANCER PLATFORM DB ---  
1. Register  
2. Login  
3. Exit  
Enter choice (1-3):
```

```

--- FREELANCER PLATFORM DB ---

1. Register
2. Login
3. Exit

Enter choice (1-3): 1
[REGISTRATION] Enter Full Name: John Doe
[REGISTRATION] Enter Email: john.doe@email.com
[REGISTRATION] Enter Password: ****
[REGISTRATION] Register as (client/freelancer): freelancer

Registration Successful! Returning to menu... ✨

```

```

--- FREELANCER PLATFORM DB ---

Enter choice (1-3): 2
[LOGIN] Enter Email: john.doe@email.com
[LOGIN] Enter Password: ****

Login Successful! Welcome John Doe ✨

```

2. Sample Output (Queries)

a) View Users

SQL

```
SELECT * FROM Users;
```

user_id	name	email	role
1	Alice	alice@mail.com	client
2	Bob	bob@mail.com	freelancer
3	Charlie	charlie@mail.com	freelancer

b) View Projects

SQL

```
SELECT * FROM Projects;
```

project_id	title	budget	client_id
1	Website Design	500	1

c) View Bids

SQL

```
SELECT * FROM Bids;
```

bid_id	project_id	freelancer_id	bid_amount
1	1	2	450
2	1	3	400

d) Join Query (Freelancers bidding on projects)

SQL

```
SELECT u.name, p.title, b.bid_amount
FROM Bids b
JOIN Users u ON b.freelancer_id = u.user_id
JOIN Projects p ON b.project_id = p.project_id;
```

name	title	bid_amount
Bob	Website Design	450
Charlie	Website Design	400

```
--- CLIENT DASHBOARD ---

--- FREELANCER PLATFORM DB ---

Enter choice (1-3): 1
[PROJECT] Enter Title: Website Design
[PROJECT] Enter Description: Build responsive website
[PROJECT] Enter Budget: 500

Project Posted Successfully! ✨
```

```
--- FREELANCER DASHBOARD ---

Available Projects:
1. Website Design | Budget: 500

Enter Project ID to Bid: 1
Enter Your Bid Amount: 400
Bid Placed Successfully!
Bid Placed Successfully!

✨
```

Summary of Key Results

The developed Freelancer Platform Database system successfully enables users to register, log in, post projects, and place bids. The system efficiently stores and retrieves data using MySQL, ensuring smooth interaction between clients and freelancers. All core functionalities were

implemented and tested successfully.

Interpretation

The results indicate that the system effectively manages relationships between users, projects, and bids. The database design ensures minimal redundancy and maintains data integrity.

Achievement of objectives

- A normalized database schema (up to 3NF) was implemented.
- Core functionalities like project posting and bidding were developed.
- Integration between frontend, backend, and database was established.

Implications

The developed system can be used as a foundation for real-world freelancer platforms. It demonstrates how database management concepts can be applied to build scalable applications.

6 Mapping POs and SDGs

6.3 Program Outcomes (POs) Mapping

Choose appropriate Program outcome and describe its relevance in implementation of your Course End Project. Delete the rows in table if any is not mapped.

Table 1: Mapping of Program Outcomes (POs) to Project Relevance

PO No.	Program Outcome	Relevance
PO1	Engineering Knowledge	Applied database concepts like normalization, SQL, and relational modeling to design the Freelancer Platform database.
PO2	Problem Analysis	Analyzed the challenges of managing freelancer-client interactions and identified efficient data handling solutions.
PO3	Design/Development of Solutions	Designed a structured database system with entities like User, Project, and Bid to solve real-world problems.
PO4	Conduct Investigations of Complex Problems	Studied existing freelancer platforms and database models to design an optimized and scalable solution.
PO5	Engineering Tool Usage	Used tools such as MySQL, Python Flask, and VS Code for implementation and testing of the system.
PO6	The Engineer and The World	Developed a system that supports the digital economy by connecting freelancers and clients.

		clients globally.
PO7	Ethics	Ensured secure handling of user data and maintained integrity and privacy in database transactions.
PO8	Individual and Collaborative Team work	Worked as a team to divide tasks such as database design, coding, and documentation.
PO9	Communication	Effectively documented the project and presented results using diagrams, reports, and explanations.
PO10	Project Management and Finance	Managed project tasks efficiently and understood financial aspects like bidding
PO11	Life-Long Learning	Gained knowledge of modern database systems

6.3 SustainableDevelopmentGoals(SDGs)Mapping

Choose most appropriate SGS(Maximum of 4) and describe its relevance in implementation of your Course End Project. Delete the rows in table if any is not mapped.

Table2:MappingofSustainableDevelopmentGoals(SDGs)toProjectRelevance

SDGNo.	Goal	Relevance
SDG4	Quality Education	The project supports SDG 4 (Quality Education) by providing practical knowledge of database systems and real-world application development. It enhances technical skills and promotes learning through hands-on implementation.
SDG5	Gender Equality	The project supports SDG 5 (Gender Equality) by providing a platform where all users can participate equally without gender discrimination. It ensures equal access to opportunities for freelancers and clients.
SDG8	Decent Work and Economic Growth	The project supports SDG 8 (Decent Work and Economic Growth) by enabling freelancing opportunities for individuals to earn income. It promotes flexible employment and contributes to economic development.
SDG9	Industry, Innovation and Infrastructure	The project supports SDG 9 (Industry, Innovation and Infrastructure) by developing a digital platform using modern technologies like databases frameworks. It promotes innovation in building scalable and efficient online systems.

7 Conclusion

The Freelancer Platform Database was successfully designed and implemented using MySQL and Python Flask, enabling efficient management of users, projects, and bids. The project demonstrated the practical application of database concepts such as ER modeling, normalization, and SQL queries, ensuring data integrity and reduced redundancy. The integration of frontend and backend components provided a functional system that allows smooth interaction between clients and freelancers.

For future improvements, the system can be enhanced by adding secure payment gateways, real-time notifications, and advanced search features. Further development can include AI-based project recommendations and improved security mechanisms to make the platform more scalable and user-friendly.

Bibliography

- 1 Official documentation of MySQL – <https://dev.mysql.com/doc/>
- 2 Flask Documentation – <https://flask.palletsprojects.com/>
- 3 W3Schools SQL Tutorial – <https://www.w3schools.com/sql/>
- 4 GeeksforGeeks (DBMS Concepts) – <https://www.geeksforgeeks.org/dbms/>
- 5 TutorialsPoint (Database Systems) – <https://www.tutorialspoint.com/dbms/>
- 6 Elmasri, R., & Navathe, S. B. *Fundamentals of Database Systems*, Pearson Education.
- 7 Silberschatz, A., Korth, H. F., & Sudarshan, S. *Database System Concepts*, McGraw-Hill.
- 8 Research papers on freelancer platforms and gig economy from IEEE Xplore and Google Scholar.
- 9 Articles on database normalization and relational models from academic journals and conference proceedings.